

Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

ICMP Blind Connection-Reset & Blind Throughput Reduction Attack Against TCP

Design Report

CSE 406 - Computer Security Sessional

Topic No.	12
Submitted by	Anika Morshed Student ID: 2105068 Diganta Saha Tirtha Student ID: 2105081
Section	Section : B1
Date	September 12, 2026

Contents

1	Overview of the Project Idea	1
1.1	Attack 1 - ICMP Blind Connection-Reset	1
1.2	Attack 2 - ICMP Blind Throughput Reduction	1
2	Network Topology, Components, and Timing Diagram	2
2.1	Network Topology	2
2.2	Components	2
2.3	Timing Diagram - Connection-Reset Attack	2
2.4	Timing Diagram - PMTU Throughput Reduction Attack	3
3	Packet Structure	3
3.1	ICMP Error Packet Structure (RFC 792)	3
3.2	Attack 1 - ICMP Type 3, Code 3 (Connection Reset)	4
3.3	Attack 2 - ICMP Type 3, Code 4 (PMTU Reduction)	4
4	Implementation Plan	5
5	Expected Outcome of a Successful Attack	6
5.1	Attack 1 - Connection Reset	6
5.2	Attack 2 - PMTU Throughput Reduction	6
6	Defense Ideas	6
7	References	6

1. Overview of the Project Idea

1.1. Attack 1 - ICMP Blind Connection-Reset

The attacker sends a spoofed **ICMP Type 3, Code 3** (*Destination Unreachable - Port Unreachable*) message addressed to the server. The ICMP payload embeds the IP and TCP headers of an existing client-server TCP connection. Because Code 3 is a hard error under RFC 1122, a vulnerable TCP stack will immediately abort the connection.

1.2. Attack 2 - ICMP Blind Throughput Reduction

The attacker sends a spoofed **ICMP Type 3, Code 4** (*Fragmentation Needed, DF Set*) message. Per RFC 1191, the ICMP payload includes a *Next-Hop MTU* field. If accepted by the server's TCP stack, the Path MTU (PMTU) cache is updated to the advertised (artificially small) value. The TCP sender then reduces its Maximum Segment Size (MSS), sending smaller segments and achieving lower throughput.

2. Network Topology, Components, and Timing Diagram

2.1. Network Topology

The lab uses three Linux network namespaces connected via a software bridge, all running inside a single Ubuntu 22.04 virtual machine.

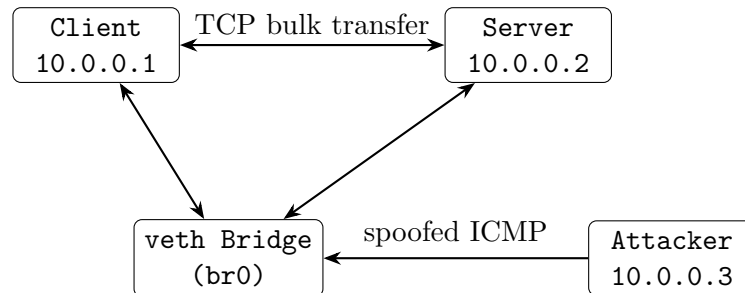


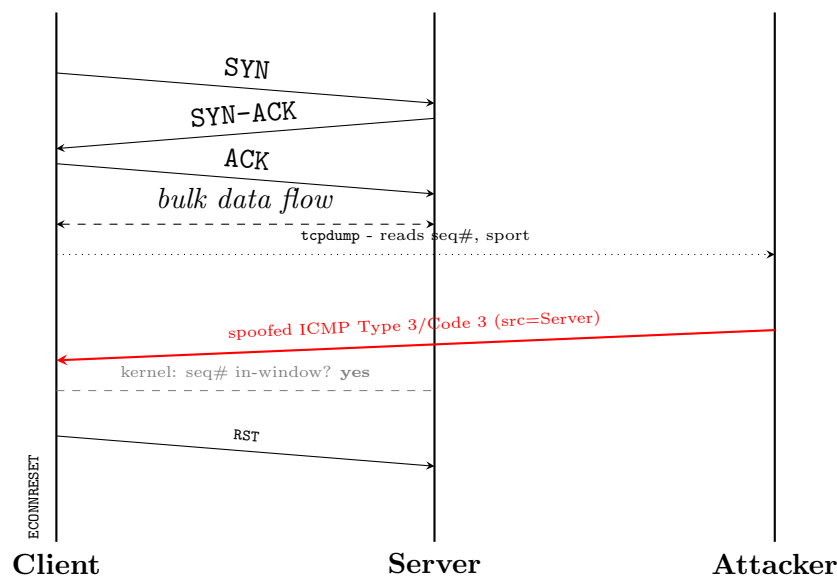
Figure 1: Network topology - three Linux namespaces on one host.

2.2. Components

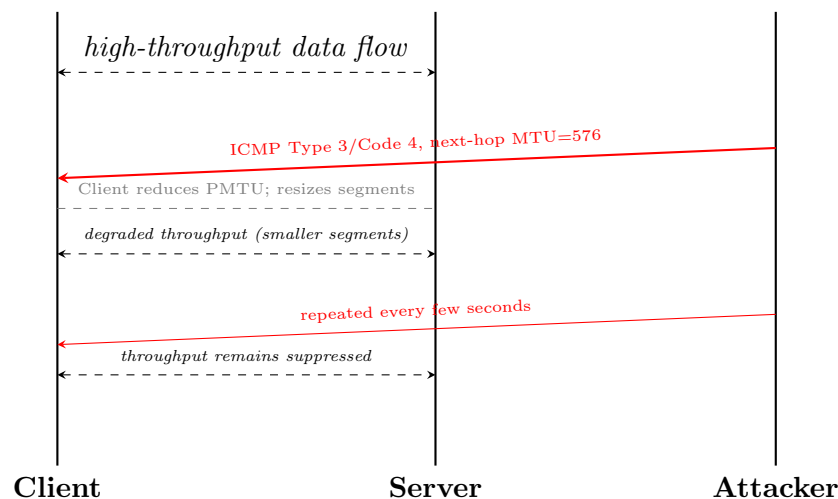
Node	IP Address	Namespace	Role
Client	10.0.0.1	client	Holds the TCP connection being targeted
Server	10.0.0.2	server	Receives spoofed ICMP; TCP state is observed here
Attacker	10.0.0.3	attacker	Crafts and injects forged ICMP packets
Bridge	—	root	br0 connects all three veth pairs

Table 1: Network components and roles

2.3. Timing Diagram - Connection-Reset Attack



2.4. Timing Diagram - PMTU Throughput Reduction Attack



3. Packet Structure

3.1. ICMP Error Packet Structure (RFC 792)

Both attack packets follow the same outer structure defined by RFC 792. An ICMP error message is carried inside an IP datagram. Its payload must contain the **original IP header** plus the **first 8 bytes of the original IP datagram** (i.e., the first 8 bytes of the TCP header).

Table 2: Spoofed ICMP packet structure (both attacks).

Layer / Header	Details / Fields
Ethernet Header	src=attacker MAC, dst=bridge MAC
IP Header	src= <i>forged</i> (Server/Router), dst=Client, proto=1 (ICMP)
ICMP Header	Type / Code / Checksum / Type-specific field
Embedded IP Header	src=Client, dst=Server, proto=6
Embedded TCP Header (8 B)	sport, dport, seq#

3.2. Attack 1 - ICMP Type 3, Code 3 (Connection Reset)

Layer	Field	Value / Notes
Outer IP	Source IP	10.0.0.1 (spoofed - attacker impersonates client)
	Destination IP	10.0.0.2 (server)
	Protocol	1 (ICMP)
ICMP	Type	3 (Destination Unreachable)
	Code	3 (Port Unreachable)
	Unused field	0
Inner IP	Source IP	10.0.0.1 (original connection client IP)
	Destination IP	10.0.0.2 (original connection server IP)
	Protocol	6 (TCP)
Inner TCP (8B)	Source Port	Client ephemeral port (e.g. 43146)
	Dest Port	Server port (9999)
	Seq Number	Must fall in server's receive window

Table 3: Field values for the connection-reset attack packet

3.3. Attack 2 - ICMP Type 3, Code 4 (PMTU Reduction)

Layer	Field	Value / Notes
Outer IP	Source IP	10.0.0.1 (spoofed)
	Destination IP	10.0.0.2
	Protocol	1 (ICMP)
ICMP	Type	3 (Destination Unreachable)
	Code	4 (Fragmentation Needed, DF Set)
	Next-Hop MTU	Advertised small MTU (e.g. 576 bytes per RFC 1191)
Inner IP	Same as above	Matches the live iperf3 connection's IP header
Inner TCP (8B)	Source Port	Client ephemeral iperf3 port
	Dest Port	5201 (iperf3)
	Seq Number	Sampled from sequence space

Table 4: Field values for the PMTU throughput-reduction attack packet

4. Implementation Plan

The implementation follows a phased approach. Each phase builds on the previous one, and no attack code is run before the infrastructure is verified.

Phase	Goal	Actions
P0	Topology setup	Create three network namespaces (<code>client</code> , <code>server</code> , <code>attacker</code>), bridge <code>br0</code> , and veth pairs. Configure <code>tc</code> mirrored traffic mirroring. Verify with <code>ping</code> and <code>ip route</code> .
P1	Baseline measurement	Run <code>iperf3</code> client→server for 30 seconds. Record average throughput, RTT, and retransmissions. Capture with <code>tcpdump</code> .
P2	Real ICMP reference	Send UDP to a closed port on server. Capture the kernel-generated ICMP Port Unreachable. Use as ground-truth for comparing Scapy output.
P3	Live State Retrieval	From the attacker namespace, run <code>tcpdump</code> on the bridge interface to capture the client's ephemeral source port and the current ACK/SEQ numbers. Parse these in Python with <code>subprocess/re</code> and pass them directly to the packet-crafting function.
P4	Reset experiment	Hold TCP connection with <code>ncat</code> . Inject ICMP Type 3 Code 3 packets with sampled seq numbers. Observe <code>ss -tn</code> and <code>tcpdump</code> on server. Classify result per the outcome table (Section 5).
P5	PMTU experiment	Run <code>iperf3</code> for 60 seconds. After 15 seconds, inject ICMP Type 3 Code 4 (MTU=576). Immediately check <code>ip route get</code> for PMTU change. Compare per-second throughput before and after injection.
P6	Metrics collection	Record: PMTU cache state, TCP segment sizes (pcap), <code>iperf3</code> throughput, RTT, retransmissions.
P7	Defense	Enable <code>tcp_mtu_probing=2</code> . Repeat P4 and P5. Record whether kernel now rejects the forged ICMP messages.

Table 5: Phased implementation plan

5. Expected Outcome of a Successful Attack

5.1. Attack 1 - Connection Reset

A successful ICMP connection-reset experiment produces the following observable chain of events:

1. The spoofed ICMP Type 3 Code 3 packet is visible in the server's `tcpdump` capture.
2. The embedded sequence number falls within the server's current receive window.
3. The server's TCP stack aborts the connection (RFC 1122 hard error behavior).
4. `ss -tn | grep 9999` returns empty on the server - the socket no longer exists.
5. The `ncat` process on the client reports a broken pipe or connection reset.

5.2. Attack 2 - PMTU Throughput Reduction

A successful PMTU attack follows this causal chain - each step must be independently verified:

1. ICMP Type 3 Code 4 packet visible in server's `tcpdump`.
2. `ip route get 10.0.0.1` on server shows `mtu 576` (PMTU cache updated).
3. TCP segment sizes in pcap decrease from ~ 1460 bytes to $\leq 576 - 40 = 536$ bytes.
4. `iperf3` per-second throughput shows measurable decrease.

6. Defense Ideas

Defense	Mechanism	Effect
Sequence number validation	Modern kernels (RFC 5961 influenced) require the embedded sequence number to be within the receive window before acting on ICMP errors.	Makes blind seq-number guessing much harder.
Ingress filtering (BCP 38)	Routers drop packets whose source IP does not belong to the originating network.	Prevents IP spoofing at the network edge entirely.

Table 6: Defenses against ICMP blind injection attacks

7. References

- [1] J. Postel, *Internet Control Message Protocol*, RFC 792, September 1981. <https://www.rfc-editor.org/rfc/rfc792>

- [2] R. Braden (Ed.), *Requirements for Internet Hosts — Communication Layers*, RFC 1122, October 1989. <https://www.rfc-editor.org/rfc/rfc1122>
- [3] J. Mogul and S. Deering, *Path MTU Discovery*, RFC 1191, November 1990. <https://www.rfc-editor.org/rfc/rfc1191>
- [4] A. Heffernan, *Protection of BGP Sessions via the TCP MD5 Signature Option*, RFC 2385, August 1998.
- [5] P. Watson, *Slipping in the Window: TCP Reset Attacks*, CanSecWest 2004.
- [6] A. Ramaiah, R. Stewart, M. Dalal, *Improving TCP's Robustness to Blind In-Window Attacks*, RFC 5961, August 2010. <https://www.rfc-editor.org/rfc/rfc5961>
- [7] F. Gont, *Deprecation of ICMP Source Quench Messages*, RFC 6633, May 2012. <https://www.rfc-editor.org/rfc/rfc6633>